

1. Python — argumentos padrão mutáveis

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista
```

```
print(adicionar(1))  
print(adicionar(2))
```

SAIDA: [1]

[1, 2]

2. Java — passagem por valor e referências

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}
```

```
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}
```

```
public static void main(String[] args) {  
    int[] v = {5, 5};  
    int n = 5;  
    zera(v, n);  
    System.out.println(v[0] + " " + n);  
}
```

SAIDA: 0 5

exer 2

3. Python — closures / late binding

```
fs = [lambda: i for i in range(3)]  
print([f() for f in fs])
```

SAIDA: [2, 2, 2]

4. C — static

```
int contador(void) {  
    static int n = 0;  
    return ++n;  
}
```

// em main:

```
contador(); contador();  
printf("%d\n", contador());
```

Saida: 3

exer3

5. Rust — ownership

```
fn dobra(v: Vec<i32>) -> Vec<i32> {  
    v.iter().map(|x| x * 2).collect()  
}
```

```
let v = vec![1, 2, 3];  
let d = dobra(v);  
println!("{:?}", v, d);
```

total = 0

SAIDA: [1, 2, 3] [2, 4, 6]

6. Python — escopo

```
def adiciona(x):  
    total = total + x
```

```
    return total  
print(adiciona(5))
```

SAIDA: 5